

UdaciHeadline: LLM Inference Optimization Project

Tools/Techniques and Experiments

Introduction: Project involves generating catchy headlines from new articles summaries using an LLM. The current inference process is sluggish, causing publication delays and driving up operational costs. In this project, UdaciHeadline, our mission is to accelerate the headline generation pipeline significantly by applying state-of-the-art LLM inference optimization techniques.

Project Summary

This project involves optimizing the inference performance of a pre-trained Large Language Model (we use Llama-3.2-1B in this project) for news headline generation. As part of the project we will:

1. **Establish a baseline** inference pipeline and measure its performance
2. Implement and evaluate architectural optimizations like **KV-caching**.
3. Apply model compression techniques like **quantization** and **pruning**.
4. Configure and benchmark **distributed inference** using Tensor and Pipeline Parallelism.
5. Apply advanced decoding mechanisms like **speculative decoding**.
6. Perform comprehensive **benchmarking and analysis** across all stages.
7. Produce a **final report** summarizing findings and trade-offs.

This document covers step 7 of the project. Steps 1-6 are implemented in Jupyter notebook

Technical Project layout

- 1) We use following LLMs as part of this project:
 - a) "meta-llama/Llama-3.2-1B-Instruct" → this is the main model used in all the experiments except for speculative decoding where we use it as draft model.
 - b) "meta-llama/Llama-3.2-3B-Instruct" → This is only used in speculative decoding experiment where it acts as target model

We used 16 bit float version of these models in all the experiments except few.

- 2) We implemented following functions. Their use is also mentioned in brief.
 - a) `load_news_dataset()` → This function downloads dataset which is in json format from Kaggle. This function then converts the dataset into pandas format which is then converted into hugging face Dataset format
 - b) `get_model_memory_footprint` → returns the memory footprint of the model
 - c) `load_model` → This function loads model and tokenizer. This function also accepts quantization config and quantize the model if required.
 - d) `convert_chat_form_tokenize` → This function converts the dataset into format suitable for chat and then applies tokenizer's chat template on it. It also performs tokenization on chat format text. It also splits dataset into train and test.
 - e) `generate_config` → This functions creates a dictionary assigning suitable values to text generation configuration parameters. We set max token generation length to 40. We also configure whether or not to use KV cache through these parameters.

- f) `base_format` → formats dataset into a list of messages suitable as input to chat template for both training and text generation task.
- g) `apply_chat_temp` → Applies tokenizer's chat template and tokenizes the input.
- h) `cuda_warmup` → This is responsible for running few test inferences on cuda device to rule out latencies attributed to model loading, kernel compilation/placement etc. This runs before actual llm inferencing takes place.
- i) `remove_punctuation` → This function removes punctuation marks from sentence.
- j) `run_exp` → This is the main wrapper function which is called by lot of experiments. It calls `load_model` function to load model and tokenizer. If model is to be pruned, it will call `prune_model_weights` to prune the model. Finally it calls `report_metrics` wrapper function which returns experiment stats in pandas dataframe format
- k) `report_metrics` → This wrapper function calls `evaluate_model` function which returns model evaluation statistics. This functions then converts the returned stats into dataframe format and sends it back to `run_exp` function
- l) `evaluate_model` → Contains the main inferencing logic and recording of all the inference statistics. We choose 100 samples for inferences and run them over 4 times resulting in total 400 runs. Samples are individually inferred. It calls `generate_headline` function for actual inferencing and records the return statistics. Once all the sample runs are over, it aggregates all the statistics and send it across to `report_metrics`.
- m) `generate_headline`→ This function does the actual inferencing and record performance stats for individual samples which it returns back to `evaluate_model`. This function performs the actual headline generation.
- n) `prune_model_weights`→ This function is responsible for pruning model weights. There is physical pruning, ie, we do not remove any model components, just zero out least significant one and make it permanent to assess impact of pruning on inferencing process
- o) `generate_headline_batch`→ This function performs batch inferencing. It does all the work listed done by `report_metrics`, `evaluate_model` and `generate_headline` functions but for batch inferences.

Experiments carried out are divided into 2 broad categories:

- 1) Experiments carried out on single GPU → This is done on ml.g5.2xlarge on AWS Sagemaker notebook instance which used A10 GPU with 24 GB VRAM. Environments with 16 GB VRAM were found unsuitable for this project as multiple experiments need to be performed which results in frequent kernel crashed plus T4 GPU is not suitable for bfloat16
- 2) Experiments carried out on multiple GPUs → These are done on ml.g4dn.12xlarge AWS Sagemaker notebook instance which uses 4 T4 GPUs. This is primarily to experiment with tensor/model/pipeline parallel cases.

Following experiments carried out both in single sample and batch format(batch size of 32):

- 1) Baseline experiment with KV cache disabled
- 2) Experiment with KV caching enabled
- 3) Pruned model experiments
- 4) Quantized model experiments
- 5) Target model baseline performance experiments in speculative decoding section
- 6) Tensor/Model parallel experiments

Actual speculative decoding is performed on single sample batch size.

In total, 24 experiments were conducted. List of experiments and their short description is provided below. We collect statistics related to latency, throughput, peak memory usage and rouge 1 score. For experiments performed in batch mode, we return latency in format `latencyperbatch/latencyperbatchsample` format and Throughput returned is per sample batch throughput. Rouge score remained almost same and very low for most of the experiments except for quantized model and pruned. This can be attributed the dataset provided. Summaries did not correlate with headlines in many of the cases. It would have been helpful if headlines correlated with short summaries and also if summaries had been more elaborate.

- 1) Baseline experiment with KV caching disabled. Latency decreases by 24% and throughput goes up by 18% when comparing single sample batch with 32 batch size inferencing
- 2) KV Caching enabled: For single sample outputs, latency and throughput almost remained same as that of baseline experiment. Significant improvements were seen in batched outputs where latency per sample decreased by 99.92% and throughput increased by approx. 11.5x. This is significant improvement. It can be attributed to two factors – utilizing KV cache and using GPU resources more efficiently. Running inferencing in batch actually super-amplified the efficiencies brought in by enabling KV cache. After this, we let KV cache enabled for remaining experiments.
- 3) Pruning with single item batch actually brought down rouge score while keeping latency and throughput roughly at baseline level. For batched outputs, it showed roughly the same improvement as in point no 2 above. Rouge 1 went up by few fractions. This could be an anomaly or perhaps zeroing out insignificant weights actually improved better generation quality.
- 4) Quantized model – single sample inference experiment showed latency going up by 1.45x over baseline and throughput reducing by 33% approx. Rouge score got degraded. For batched output it showed similar performance as with experiments in points 2 and 3 above though with lower individual model footprint. Throughput went down by 40% approx. when compared with its counterparts in points 2 and 3 above. Latency per sample also showed 0.02 second increase which is approx. 66% higher than its counterpart experiments.. Nevertheless, for the model size footprint, it shows great improvements over baseline model with cache disabled. Slower performance compared to the counterparts can be attributed to the quant/dequant operations pverhead that happens every time inference is performed. It also recovers rouge 1 score by few points in batched mode which is comparable to its counterparts
- 5) Speculative Decoding: Target model baseline(batched and non-batched) shows performance degradation when compared to quantized model. Latency and throughputs became almost half of its counterparts in points 2 and 3 above. This may be due to larger model size(we used 16 bit 3B model which adds roughly 3.36 GB over 1B model) and hence FLOPS became relatively expensive.

Actual speculation decoding process was done in single sample batched mode. We saw performance degradation here compared to previous experiments with respect to latency and throughput. While draft model did help reduce the inference time for target model however, the operations involved in speculative decoding slowed down the gains plus target model is approx 3x the draft model which also would have slowed down the process(despite of number of inference cycles going down for target model). We

performed lookahead of 10 tokens with draft model using 400 samples → this will involve multiple draft model runs compared to previous experiments plus huge target model size + speculative decoding process overhead. Perhaps when run at a bigger scale and using larger lookahead draft tokens and using target model of same size as that of draft model and when run in multi-GPU environment, this process is likely to show improvements. This is a promising process improvement which is already showing results at enterprise level.

Following functions are used in speculative decoding process:

- 1) `report_spec_metrics` → It calls `eval_spec` function and collects the performance stats and return it back in pandas dataframe format.
 - 2) `eval_spec` → Performs cuda warm up runs, calls `generate_spec_headline` function and collects performance stats for individual samples. It then aggregates all the stats and sends it back to `report_spec_metrics` function
 - 3) `generate_spec_headline` → it calls `speculate` function which returns `delta_draft_toks`, and `delta_target_toks`. Then it calls `match` function which compares `delta_draft_toks`, with `delta_target_toks` and returns the first 'n' matching tokens between them. These agreed upon tokens are then appended to original prompt and passed to `speculate` function following which again a match is performed. This cycle continues till either or both of the two conditions are met : Model detects end of text token or agreed tokens list becomes greater than or equal to 40 tokens. It is because this is the max token generated length we have been following so far in this project.
 - 4) `Speculate` → This function passes prompt through draft model generating at the most 10 new tokens. Newly generated delta tokens are appended to original prompt and then passed through target model. From this we get `delta_draft_toks` and `delta_target_toks` which represents additional tokens generated by draft model and matching number of tokens towards the end of output generated by target model except last position which is returned back to `generate_spec_headline` function.
-
- 6) Model loading with "balanced" and "auto" device map : Batched and non-batched – both showed degraded performance with respect to other non-speculative experiments. Batched experiments show considerable improvement over single sample inferences but still far from its counterpart experiments
 - 7) Model loading with "balanced" and "auto" device map with both model and data loader managed by accelerator: Same performance as in point no 6 above. Model loaded in point 6 also have their device managed by accelerator. But here we made it explicit for both model and data placement.
 - 8) Pipeline parallel and tensor parallel experiments using deepspeed and torchrun : All these experiments were carried out using single sample inferencing with 400 datapoints. Pipeline parallel experiment showed better latency and throughput figures compared to tensor parallel experiments. Tensor parallel performed almost at the same level as model loaded with "balanced" and "auto" device map options. Pipeline parallel experiments performed better than tensor parallel experiments.
 - 9) All the experiments carried out in 6, 7, and 8 above showed worse performance than experiments carried out in single GPU environment, Perhaps, all the experiments carried out in 6 and 7 and 8 are not suited for small scale inferencing that we are doing with limited data samples. It is like moving a small wood log with a complex and heavy machinery meant for moving big earth rock like material. Let us say in an office, one

person is sufficient to handle all the load still with lot of free time. Now, if I employ 3 more people and make the task complicated where all four must work simultaneously to achieve a small target. This overhead is not required and makes the entire process inefficient.

After analysing all the experiments outcome following conclusions can be made:

- 1) Batched operations along with KV Caching produce significant improvement over single sample operations with respect to latency and throughput
- 2) Almost all the experiments show similar rouge 1 scores except for few pruning and quantization experiment where rouge 1 score is expected to degrade. From this we can say, that given current experiments setup, rouge 1 score is not the deciding factor in choosing best over all. Almost all the models have same model footprint except for speculative decoding where we have bigger model footprint(because we have a big and a relatively not so big model in memory). So, peak mem is also ruled out. Latency and throughputs have to be deciding factor
- 3) All pipeline/model/tensor parallel experiments are not suited for small scale inferencing task with relatively small 1B parameter model(is it small? Huh!!).
- 4) Looking at stats in performance table then, it seems that experiment with name "KV Caching Batch: 32" is the best setup given current inference scale and model size. It performs inferencing in batches of 32 samples with KV Caching On using base 1B parameter model and has batch/sample latency of 0.94/0.03s and throughput of 795 samples per sec